

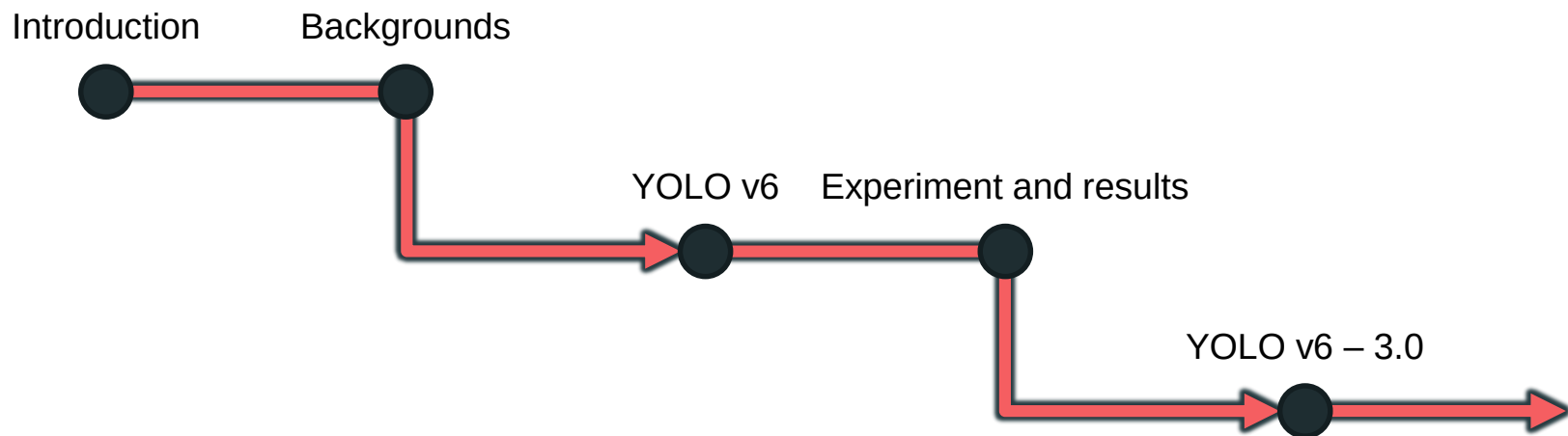
YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications

arxiv 2022

이미지처리팀

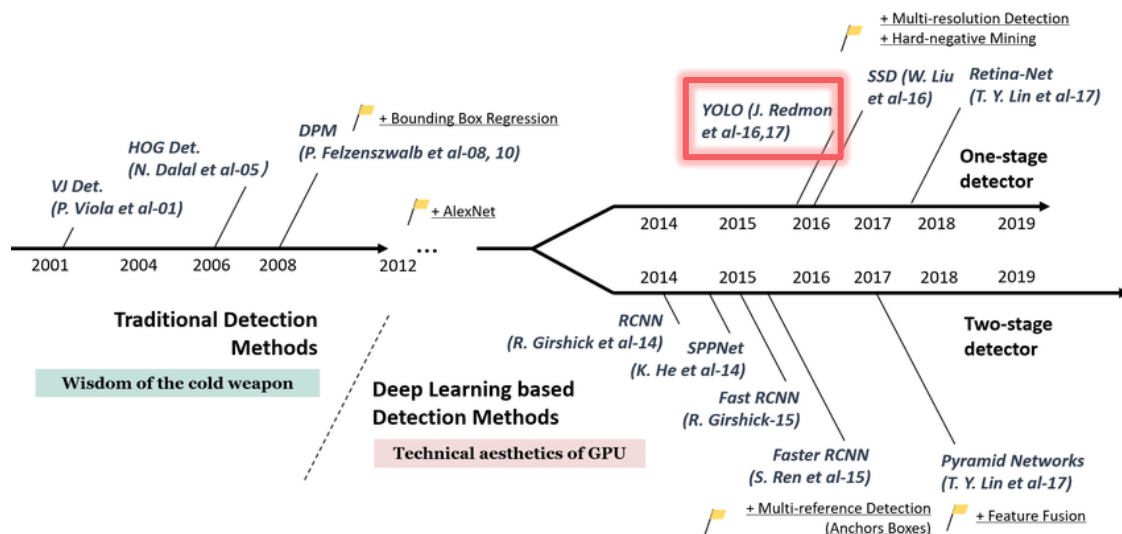
강인하, 김현진, 이찬혁, 최승준, 현청천

CONTENTS

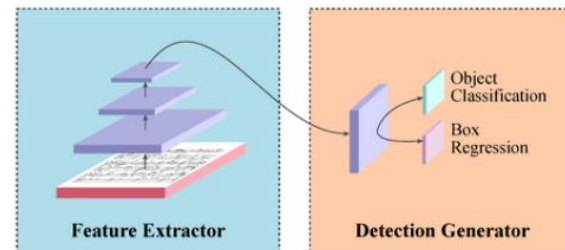


Introduction

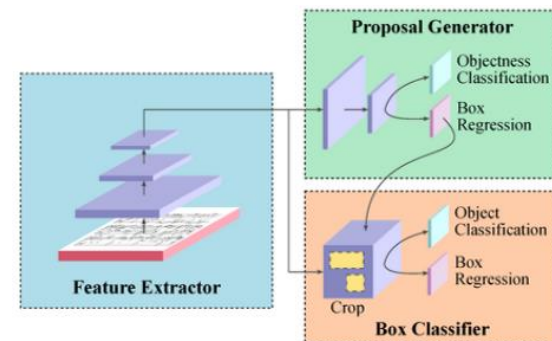
1-stage detection



Object Detection Milestones



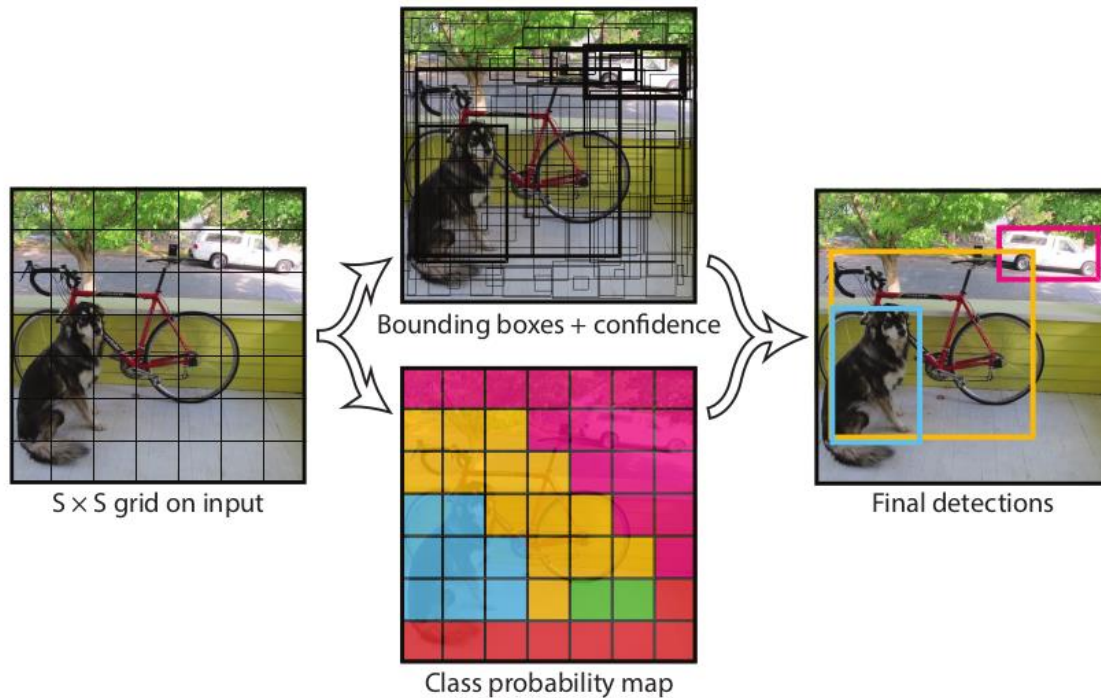
1-stage detection



2-stage detection

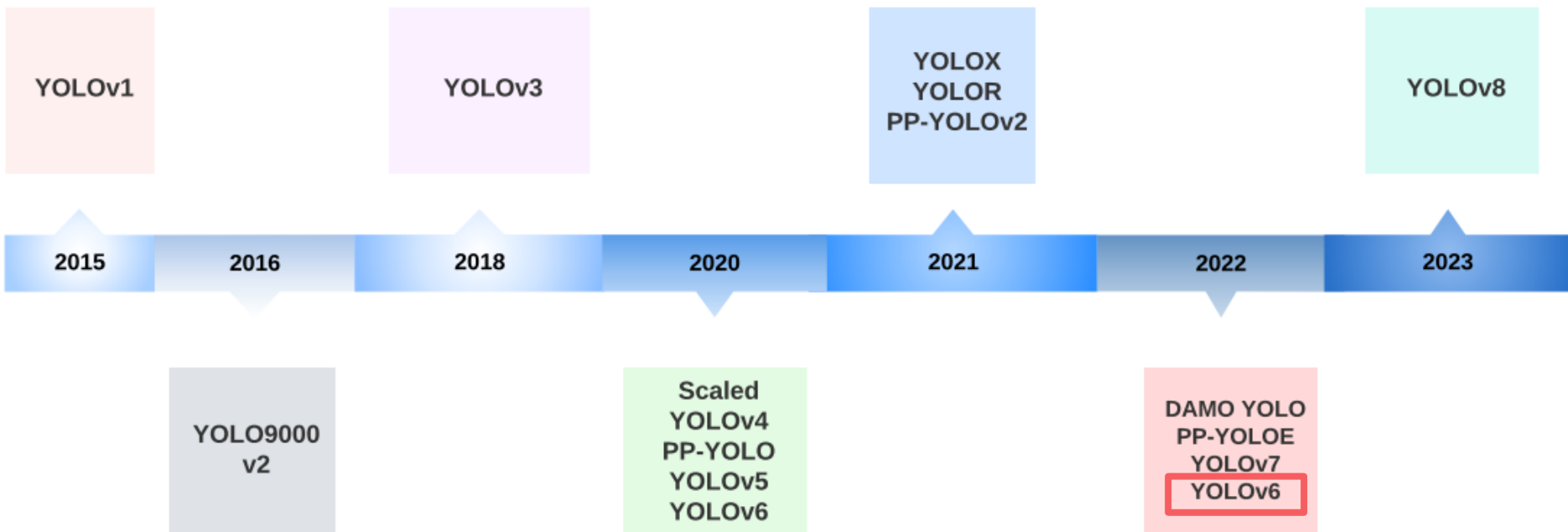
Introduction

Working Principle of YOLO



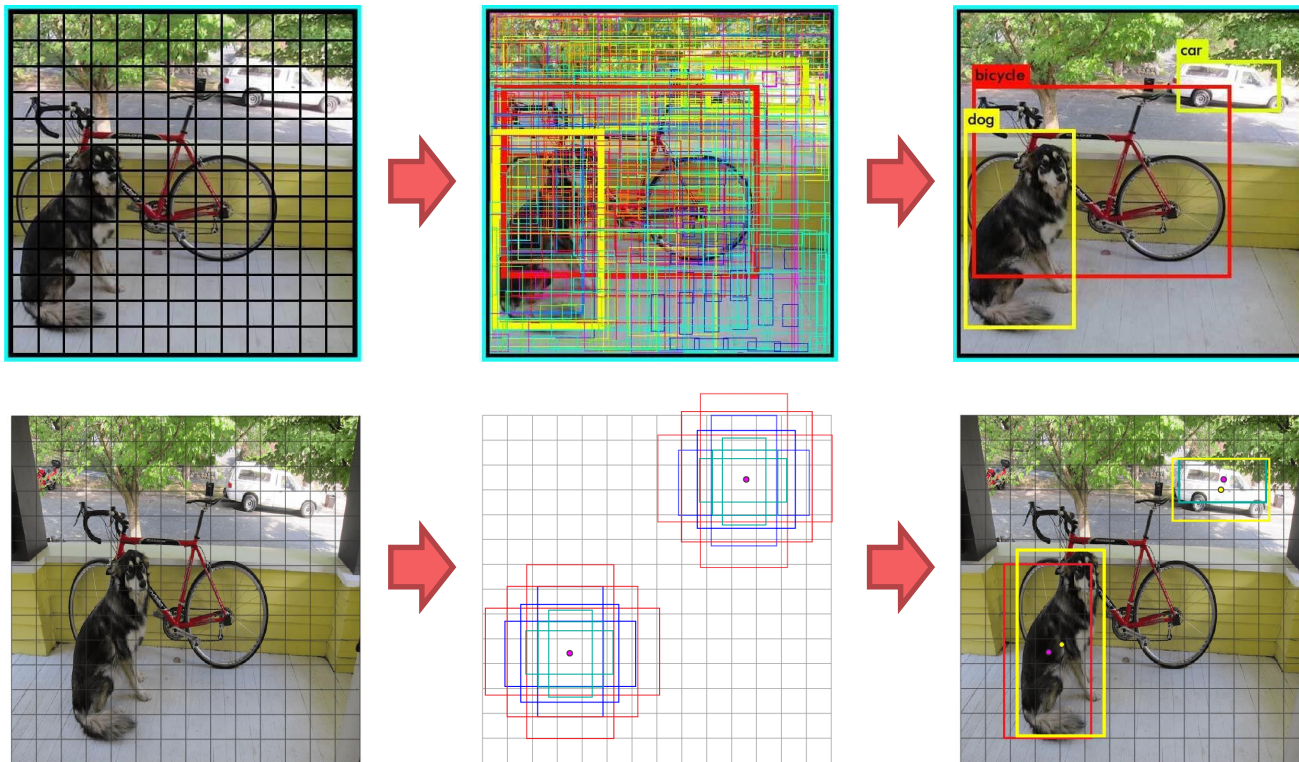
Introduction

Development of YOLO



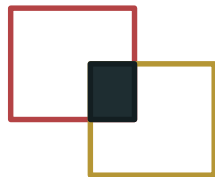
Backgrounds

Anchor Box

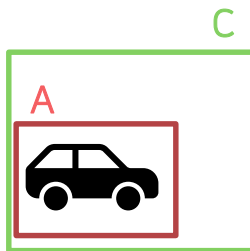


Backgrounds

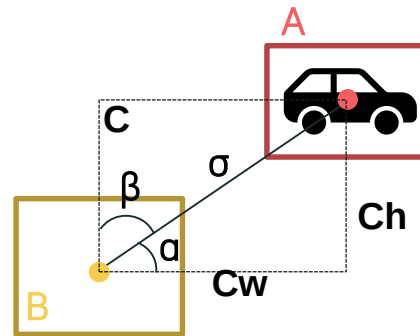
IoU (Intersection Over Union)



$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



$$\text{GloU} = \text{IoU} - \frac{|C(A \cup B)|}{|C|}$$



$$\text{SloU} = 1 - \text{IoU} - \frac{\Delta + \Omega}{2}$$

- Angle cost Δ
- Shape cost Ω
- Distance cost Δ (use shape cost)
- IoU cost

Angle cost

$$\Delta = 1 - 2 * \sin^2 \left(\arcsin(x) - \frac{\pi}{4} \right),$$

where

$$x = \frac{C_h}{\sigma} = \sin(\alpha)$$

$$\sigma = \sqrt{(b_x^{gt} - b_x)^2 + (b_y^{gt} - b_y)^2}$$

$$c_h = \max(b_{cy}^{gt}, b_{cy}) - \min(b_{cy}^{gt}, b_{cy})$$

Shape cost

$$\Delta = \sum_{t=x,y} (1 - e^{-\gamma \rho_t}),$$

where

$$\rho_x = \left(\frac{b_{cx}^{gt} - b_{cx}}{c_w} \right)^2, \rho_y = \left(\frac{b_{cy}^{gt} - b_{cy}}{c_h} \right)^2, \gamma = 2 - \Delta$$

Distance cost

$$\Omega = \sum_{t=x,y} (1 - e^{-\omega_t})^{\beta}$$

where

$$\omega_x = \frac{|w - w^{gt}|}{\max(w, w^{gt})}, \omega_y = \frac{|h - h^{gt}|}{\max(h, h^{gt})}$$

Backgrounds

PAN

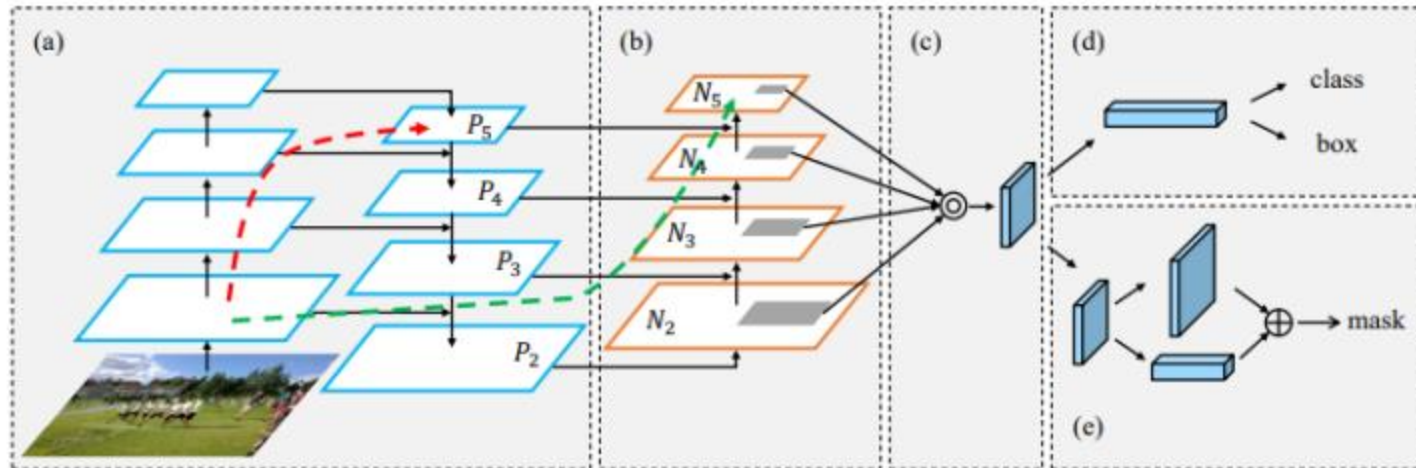
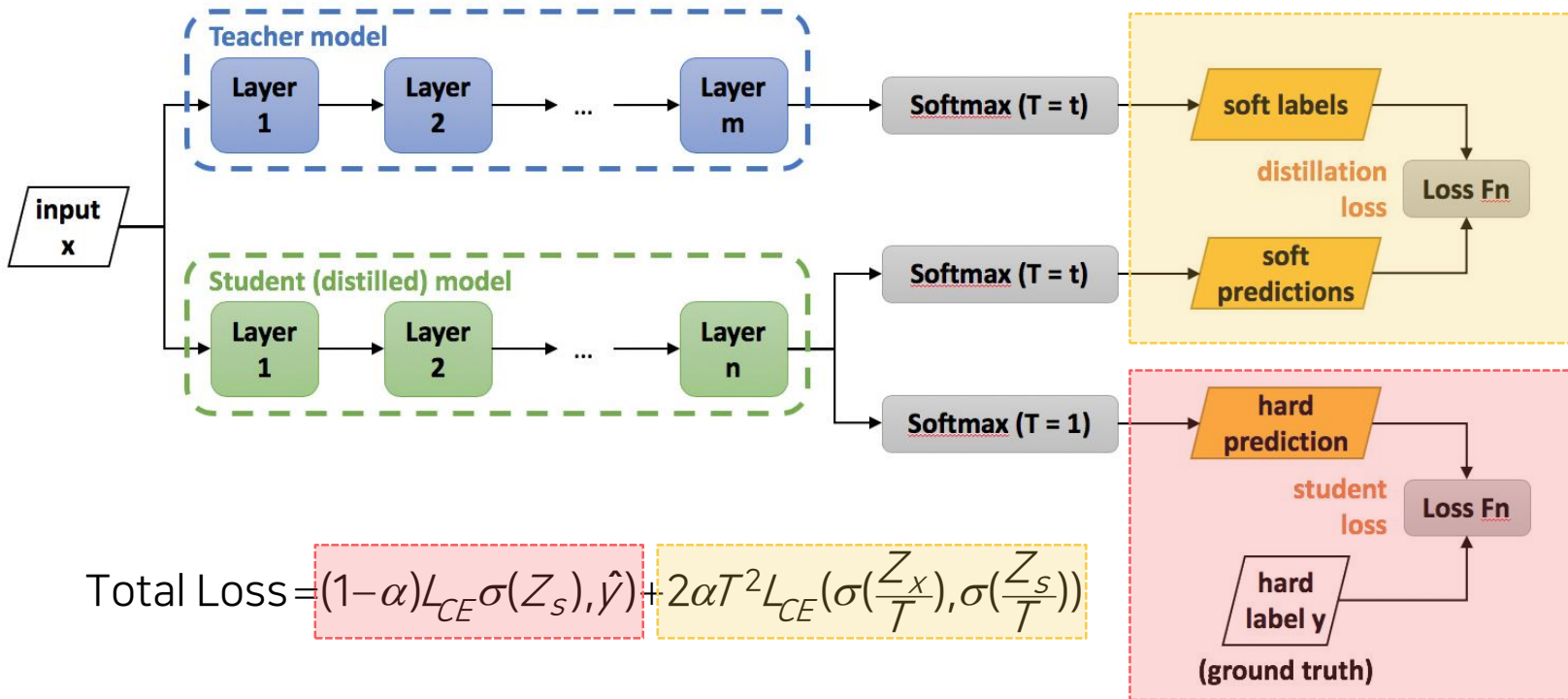


Figure 1. Illustration of our framework. (a) FPN backbone. (b) Bottom-up path augmentation. (c) Adaptive feature pooling. (d) Box branch. (e) Fully-connected fusion. Note that we omit channel dimension of feature maps in (a) and (b) for brevity.

Backgrounds

Self-distillation



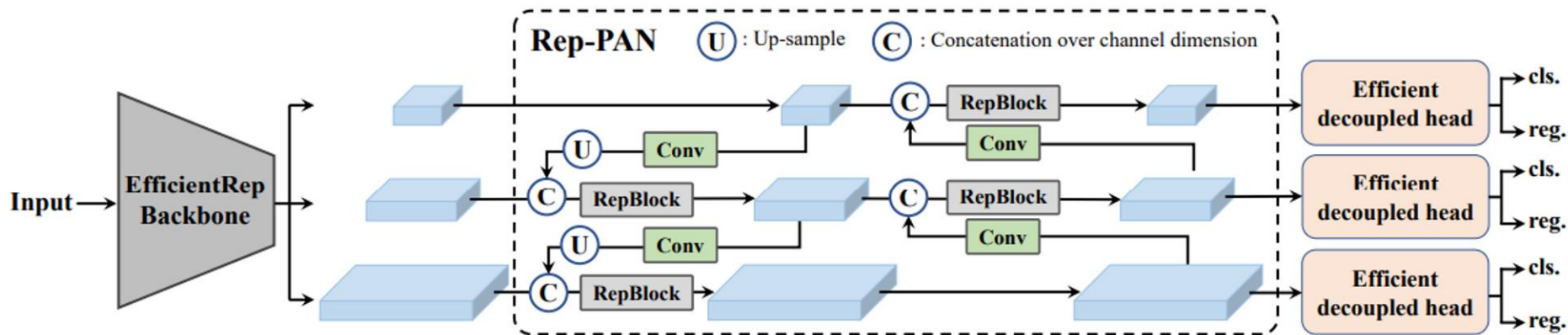


Question

YOLO v6

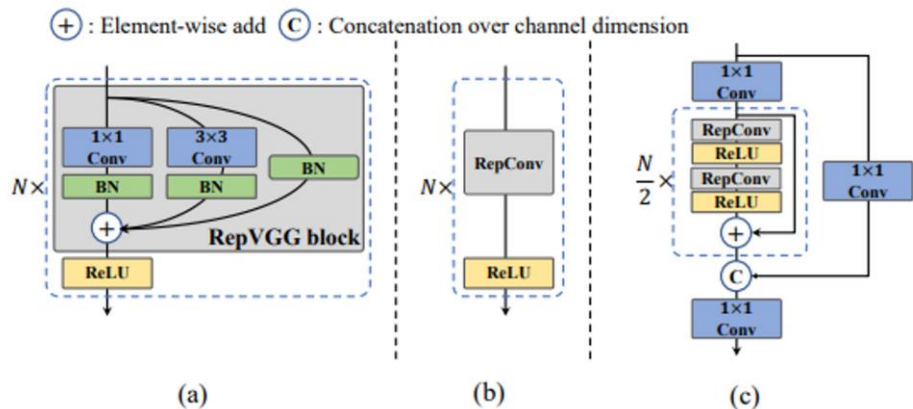
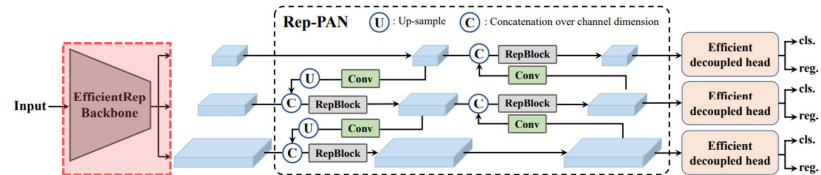
Purpose of YOLO v6

YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications



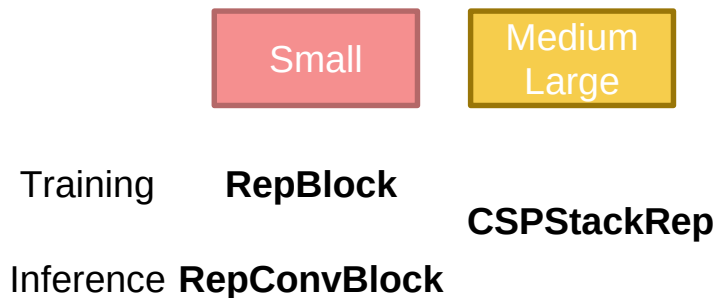
YOLO v6

Efficient Backbone



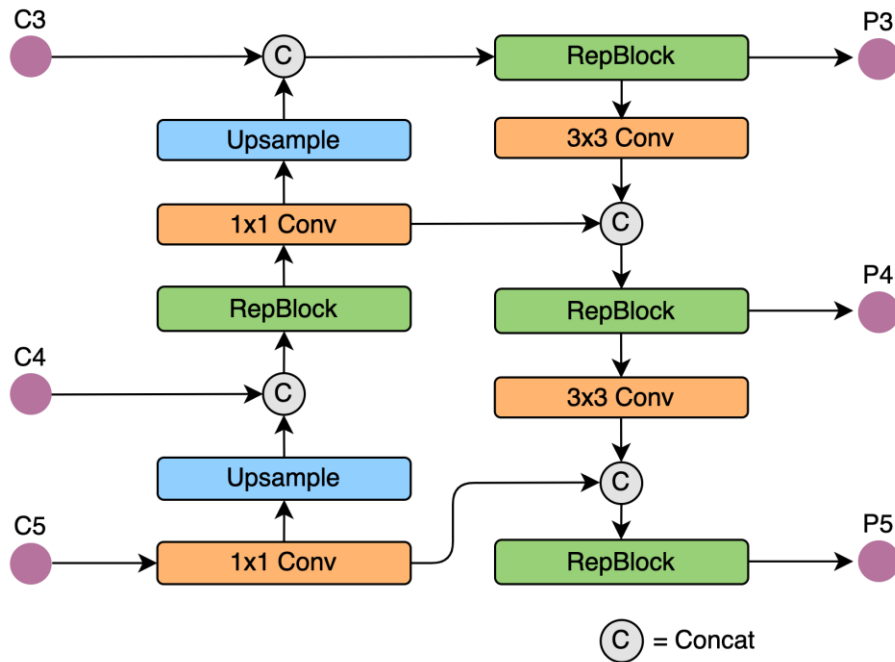
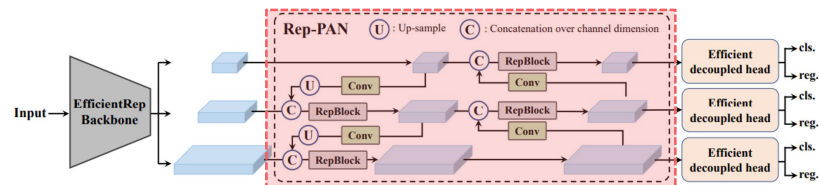
RepBlock RepConvBlock CSPStackRep

RepVGG → RepConv



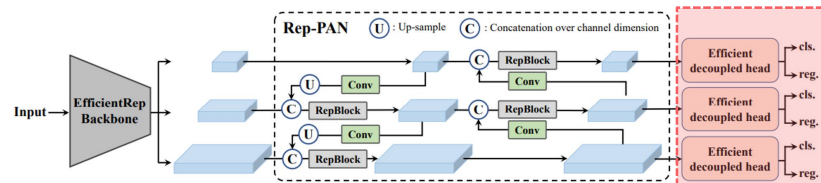
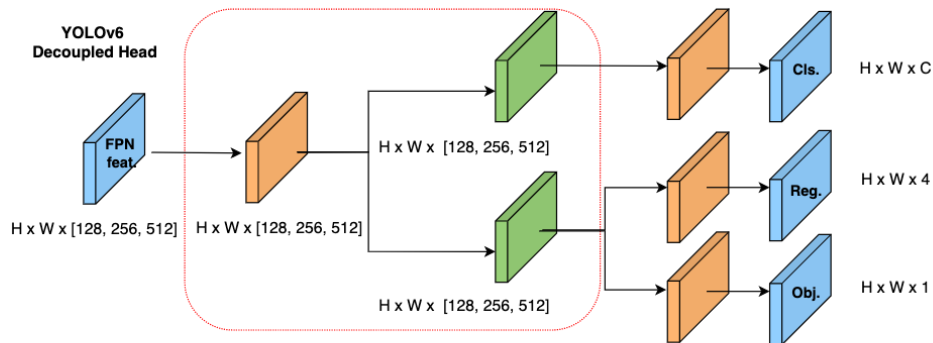
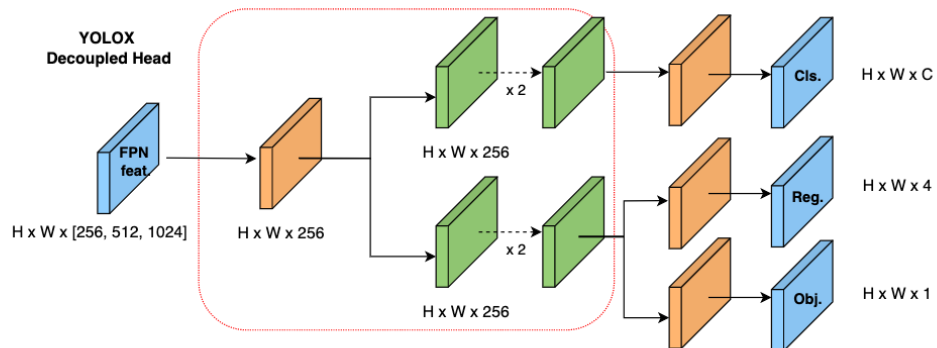
YOLO v6

Rep-PAN Neck



YOLO v6

Efficient Decoupled Head



YOLO v6

Loss function

$$L_{\text{det}} = L_{\text{cls}} + \lambda L_{\text{reg}} + \mu L_{\text{obj}}$$

↓
classification

- Focal loss
- Quality Focal loss(QFL)
- VariFocal loss(VFL)
- Poly Loss

Model	Classification Loss	AP ^{val}
YOLOv6-N	Focal Loss [22]	35.0%
	Poly Loss [17]	34.0%
	QFL [20]	35.4%
	VFL [50]	35.2%
YOLOv6-S	Focal Loss [22]	42.9%
	Poly Loss [17]	41.5%
	QFL [20]	43.1%
	VFL [50]	43.2%
YOLOv6-M	Focal Loss [22]	48.0%
	Poly Loss [17]	46.9%
	QFL [20]	48.0%
	VFL [50]	48.1%

Table 8: Ablation study on classification loss functions.

YOLO v6

Loss function

$$L_{\text{det}} = L_{\text{cls}} + \lambda L_{\text{reg}} + \mu L_{\text{obj}}$$

regression

<box regression>

- GIoU → extra models
- SIoU → N, T models
- CIoU

<probability regression>

- DFL → M, L models

Model	Loss	AP ^{val}
YOLOv6-N	GIoU [35]	35.1%
	CIoU [52]	35.1%
	SIoU [8]	35.5%
YOLOv6-S	GIoU [35]	43.1%
	CIoU [52]	43.1%
	SIoU [8]	43.3%
YOLOv6-M	GIoU [35]	48.2%
	CIoU [52]	48.3%
	SIoU [8]	48.1%

Table 9: Ablation study on IoU-series box regression loss functions. The classification loss is VFL [50].

Method	Loss	AP ^{val}	FPS (bs=32)
YOLOv6-N	w/o	35.0%	1226
	DFL [20]	35.2%	1022
	DFLv2 [19]	35.2%	819
YOLOv6-S	w/o	42.9%	486
	DFL [20]	43.0%	461
	DFLv2 [19]	43.0%	422
YOLOv6-M	w/o	48.0%	233
	DFL [20]	48.2%	236
	DFLv2 [19]	48.3%	226

Table 10: Ablation study on probability loss functions.

YOLO v6

Loss function

$$L_{\text{det}} = L_{\text{cls}} + \lambda L_{\text{reg}} + \mu L_{\text{obj}}$$

↓
object

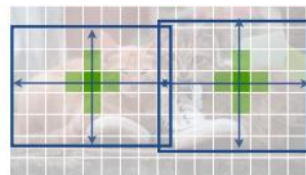
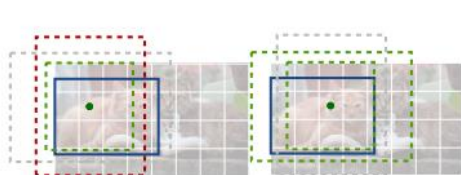
- FCOS
- Negative effect only (discarded)

Method	Object Loss	AP ^{val}
YOLOv6-N	✗	35.0%
	✓	33.9%
YOLOv6-S	✗	42.9%
	✓	41.4%
YOLOv6-M	✗	48.0%
	✓	46.5%

Table 11: Effectiveness of object loss.

YOLO v6

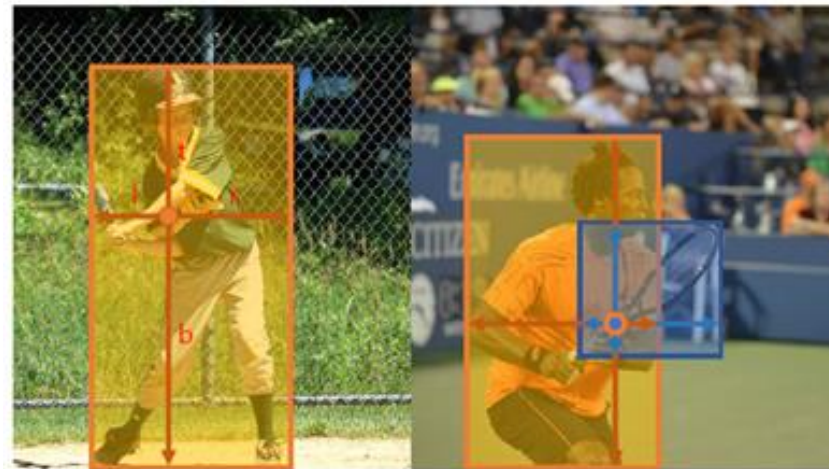
Anchor-free



(a) Standard anchor based detection. Anchors count as **positive** with an overlap $IoU > 0.7$ to any **object**, **negative** with an overlap $IoU < 0.3$, or are **ignored** otherwise.

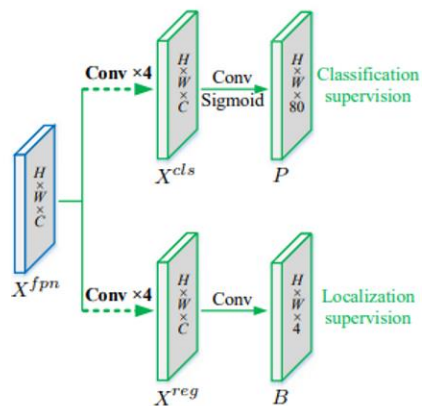
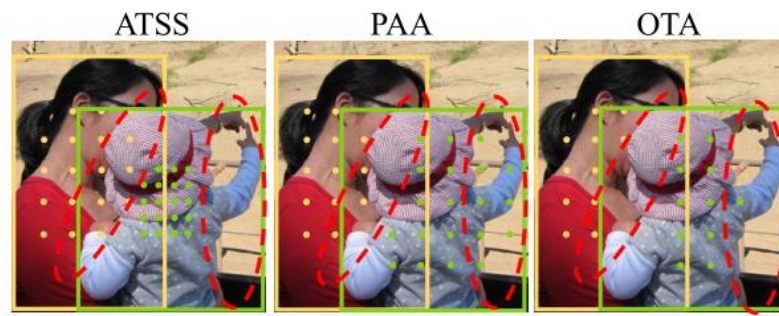
(b) Center point based detection. The **center pixel** is assigned to the **object**. Nearby points have a reduced negative loss. Object size is regressed.

Figure 3: Different between anchor-based detectors (a) and our center point detector (b). Best viewed on screen.

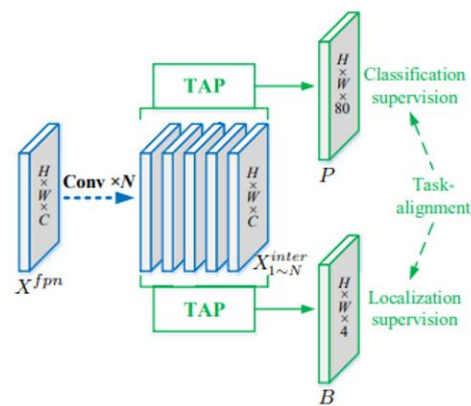


YOLO v6

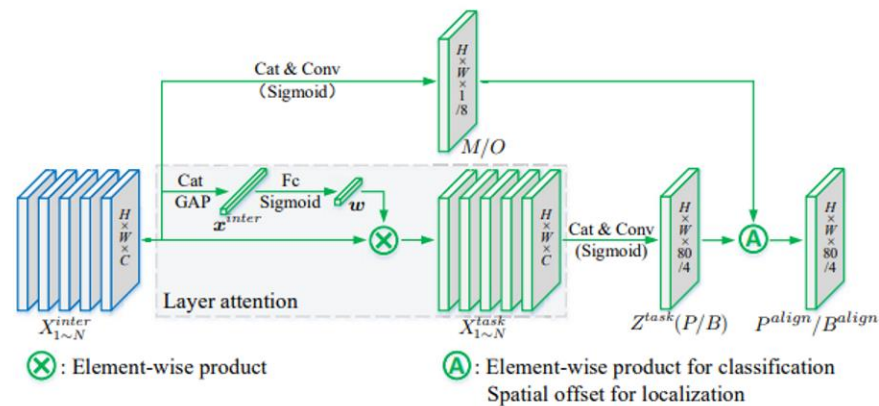
Label Assignment



(a) Parallel head



(b) Task-aligned head (T-Head)



(c) Task-aligned predictor (TAP)

Figure 3. Comparison between the conventional parallel head and the proposed T-Head.

YOLO v6

Industry-Handy improvements: Self-distillation

Training epochs

- Empirical results
- 300~400 epochs → better convergence

Quantization

- Speed 향상을 위해 quantization 기법 적용
- PTQ와 QAT 방법 비교
PTQ (성능향상 없음) → QAT 채택

Self-distillation

- Computational cost를 유지하면서 accuracy를 증가시키기 위해 도입
- KL-divergence를 이용한 self-distillation
- $L_{total} = L_{det} + \alpha L_{KD}$

Gray-border

- Half-stride gray border 추가
- 가장자리 근처 객체 탐지에 도움

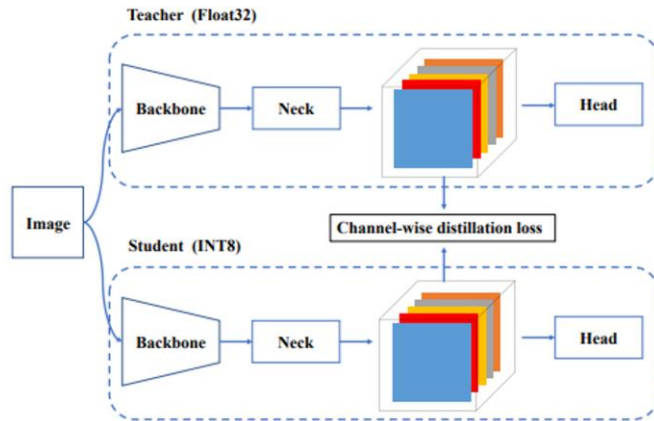
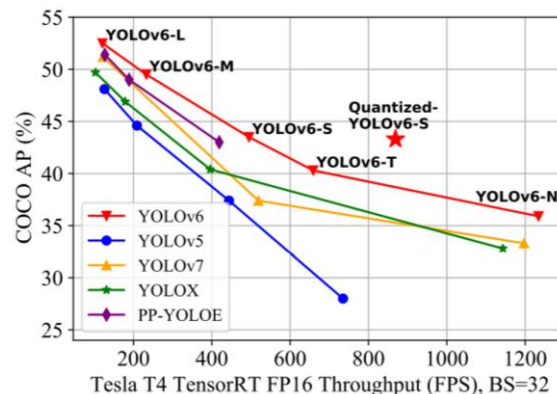
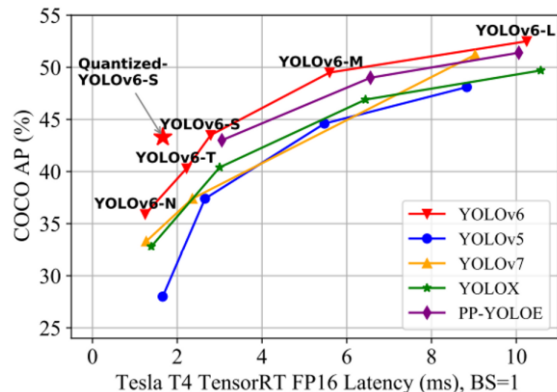


Figure 5: Schematic of YOLOv6 channel-wise distillation in QAT.

A large red square with a white border, centered on a white background. The word "Question" is written in white text in the center of the square.

Question

Experiment and results vs YOLO series



Method	Input Size	AP ^{val}	AP ^{val} ₅₀	FPS (bs=1)	FPS (bs=32)	Latency (bs=1)	Params	FLOPs
YOLOv5-N [10]	640	28.0%	45.7%	602	735	1.7 ms	1.9 M	4.5 G
YOLOv5-S [10]	640	37.4%	56.8%	376	444	2.7 ms	7.2 M	16.5 G
YOLOv5-M [10]	640	45.4%	64.1%	182	209	5.5 ms	21.2 M	49.0 G
YOLOv5-L [10]	640	49.0%	67.3%	113	126	8.8 ms	46.5 M	109.1 G
YOLOX-Tiny [7]	416	32.8%	50.3%*	717	1143	1.4 ms	5.1 M	6.5 G
YOLOX-S [7]	640	40.5%	59.3%*	333	396	3.0 ms	9.0 M	26.8 G
YOLOX-M [7]	640	46.9%	65.6%*	155	179	6.4 ms	25.3 M	73.8 G
YOLOX-L [7]	640	49.7%	68.0%*	94	103	10.6 ms	54.2 M	155.6 G
PPYOLOE-S [45]	640	43.1%	59.6%	327	419	3.1 ms	7.9 M	17.4 G
PPYOLOE-M [45]	640	49.0%	65.9%	152	189	6.6 ms	23.4 M	49.9 G
PPYOLOE-L [45]	640	51.4%	68.6%	101	127	10.1 ms	52.2 M	110.1 G
YOLOv7-Tiny [42]	416	33.3%	49.9%*	787	1196	1.3 ms	6.2 M	5.8 G
YOLOv7-Tiny [42]	640	37.4%	55.2%*	424	519	2.4 ms	6.2 M	13.7 G*
YOLOv7 [42]	640	51.2%	69.7%	110	122	9.0 ms	36.9 M	104.7 G
YOLOv6-N	640	35.9%	51.2%	802	1234	1.2 ms	4.3 M	11.1 G
YOLOv6-T	640	40.3%	56.6%	449	659	2.2 ms	15.0 M	36.7 G
YOLOv6-S	640	43.5%	60.4%	358	495	2.8 ms	17.2 M	44.2 G
YOLOv6-M [‡]	640	49.5%	66.8%	179	233	5.6 ms	34.3 M	82.2 G
YOLOv6-L-ReLU [‡]	640	51.7%	69.2%	113	149	8.8 ms	58.5 M	144.0 G
YOLOv6-L [‡]	640	52.5%	70.0%	98	121	10.2 ms	58.5 M	144.0 G

Experiment and results

Ablation Study: Backbone & Neck

Models	Block	CC	AP ^{val}	FPS (bs=32)	Params	FLOPs
YOLOv6-N	RepBlock	-	35.2%	1237	4.3M	11.1G
	CSPStackRep Block	1/2	32.7%	1257	2.3M	5.6G
YOLOv6-S	RepBlock	-	43.2%	499	17.2M	44.2G
	CSPStackRep Block	1/2	43.4%	511	11.5M	27.7G
YOLOv6-M	RepBlock	-	47.9%	137	67.1M	175.6G
	CSPStackRep Block	2/3	48.1%	237	34.3M	82.2G
	CSPStackRep Block	1/2	47.3%	237	27.7M	68.4G
YOLOv6-L	CSPStackRep Block	2/3	50.1%	149	54.7M	142.7G
	CSPStackRep Block	1/2	50.1%	151	58.5M	144.0G

Table 2: Ablation study on backbones and necks. YOLOv6-L here is equipped with ReLU.

Width	Depth	AP ^{val}	FPS (bs=32)	Params	FLOPs
[192, 384, 768]	5	50.8%	123	58.6 M	144.7 G
[128, 256, 512]	12	51.0%	122	58.5 M	144.0 G

Table 3: Ablation study on the neck settings of YOLOv6-L. SiLU is selected as the activation function.

Experiment and results

Ablation Study: Combination, Activation function, Label assignment

Model	Conv.	Act.	AP ^{val}	FPS (bs=32)
YOLOv6-N	Conv	SiLU	36.6%	963
	RepConv	SiLU	36.5%	971
	Conv	ReLU	34.8%	1246
	RepConv	ReLU	35.2%	1233
	Conv	LReLU	35.4%	983
	RepConv	LReLU	35.6%	975
YOLOv6-M	Conv	SiLU	48.9%	180
	RepConv	SiLU	48.9%	180
	Conv	ReLU	47.7%	235
	RepConv	ReLU	48.1%	236
	Conv	LReLU	48.0%	185
	RepConv	LReLU	48.1%	187

Table 4: Ablation study on combinations of different types of convolutional layers (denoted as Conv.) and activation layers (denoted as Act.).

Method	AP ^{val}
ATSS [51]	32.5%
SimOTA [7]	34.5%
TAL [5]	35.0%
DW [18]	33.4%
ObjectBox [48]	30.1%

Table 6: Comparisons of label assignment methods.

Experiment and results

Ablation Study: Loss function

Model	Classification Loss	AP ^{val}
YOLOv6-N	Focal Loss [22]	35.0%
	Poly Loss [17]	34.0%
	QFL [20]	35.4%
	VFL [50]	35.2%
YOLOv6-S	Focal Loss [22]	42.9%
	Poly Loss [17]	41.5%
	QFL [20]	43.1%
	VFL [50]	43.2%
YOLOv6-M	Focal Loss [22]	48.0%
	Poly Loss [17]	46.9%
	QFL [20]	48.0%
	VFL [50]	48.1%

Table 8: Ablation study on classification loss functions.

Model	Loss	AP ^{val}
YOLOv6-N	GIoU [35]	35.1%
	CIoU [52]	35.1%
	SIoU [8]	35.5%
YOLOv6-S	GIoU [35]	43.1%
	CIoU [52]	43.1%
	SIoU [8]	43.3%
YOLOv6-M	GIoU [35]	48.2%
	CIoU [52]	48.3%
	SIoU [8]	48.1%

Table 9: Ablation study on IoU-series box regression loss functions. The classification loss is VFL [50].

Experiment and results

Ablation Study: Loss function

Method	Loss	AP ^{val}	FPS (bs=32)
YOLOv6-N	w/o	35.0%	1226
	DFL [20]	35.2%	1022
	DFLv2 [19]	35.2%	819
YOLOv6-S	w/o	42.9%	486
	DFL [20]	43.0%	461
	DFLv2 [19]	43.0%	422
YOLOv6-M	w/o	48.0%	233
	DFL [20]	48.2%	236
	DFLv2 [19]	48.3%	226

Table 10: Ablation study on probability loss functions.

Method	Object Loss	AP ^{val}
YOLOv6-N	X	35.0%
	✓	33.9%
YOLOv6-S	X	42.9%
	✓	41.4%
YOLOv6-M	X	48.0%
	✓	46.5%

Table 11: Effectiveness of object loss.



Question

YOLO v6 3.0

Performance comparison

- Neck에 Bi-directional Connection 적용
- AAT(anchor-aided training) 전략 적용

- Deepen YOLO v6
- 새로운 Self-distillation 전략

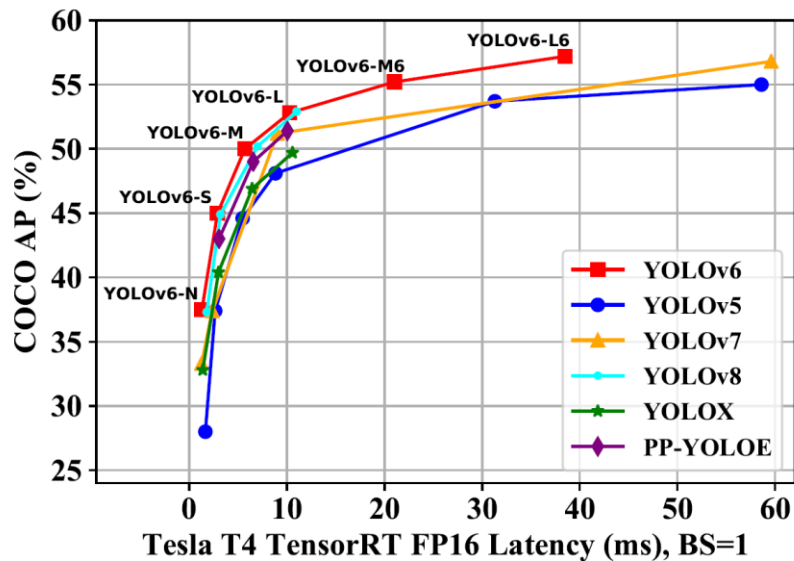
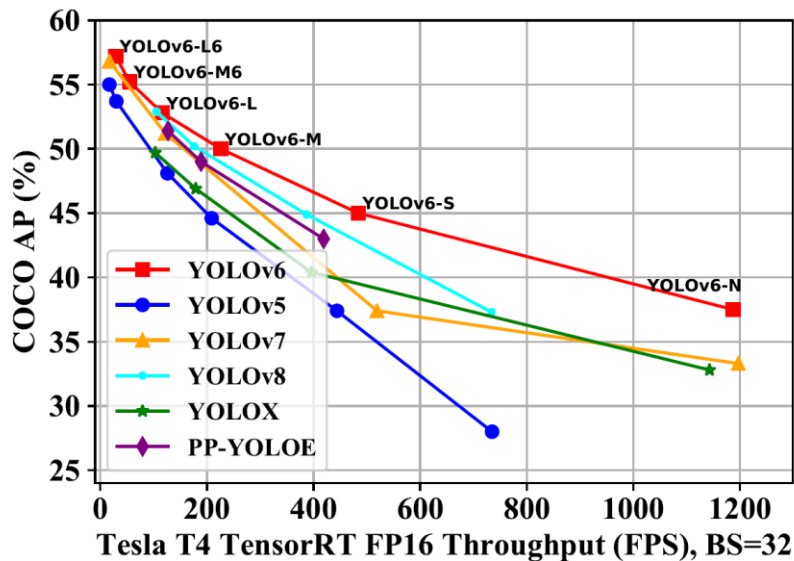
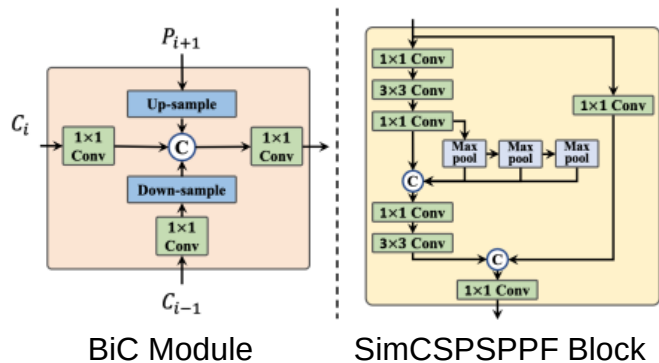


Figure 1: Comparison of state-of-the-art efficient object detectors. Both latency and throughput (at a batch size of 32) are given for a handy reference. All models are test with TensorRT 7.

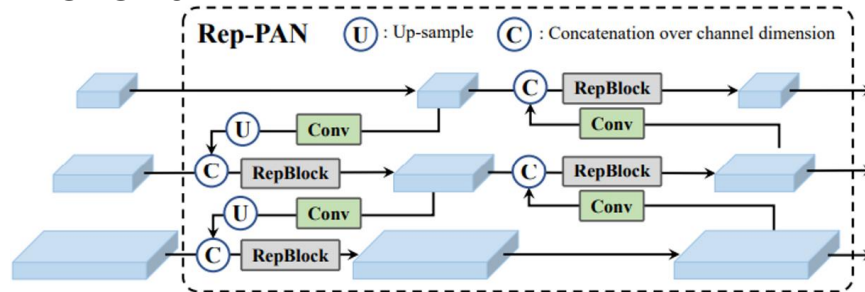
YOLO v6 3.0

Enhanced Characterization Capabilities

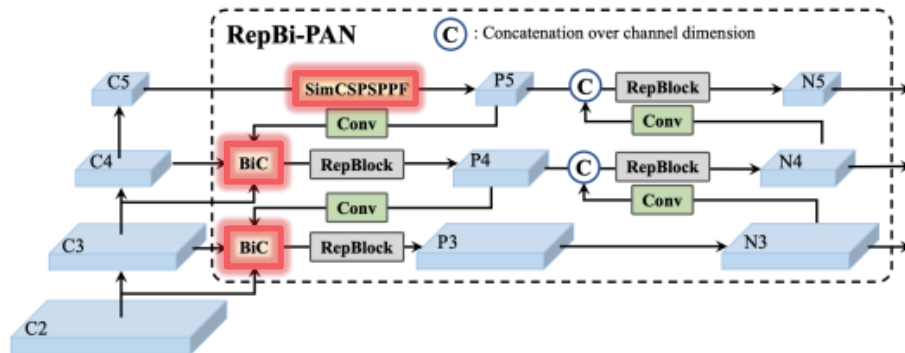
- RepBi-PAN:
localization signal을 augment하기 위해 개선한 구조
(computation cost 증가 없이)
- BiC: low-level feature를 위한 하향식+상향식 모듈
→ multi-scale feature fusion
- SimCSPSPPF: YOLO v7에서 썼던 SimSPPCSPC block 개선
representational ability를 강화한 방법



YOLO v6



YOLO v6-v3.0



YOLO v6 3.0

AAT (Anchor-Aided Training)

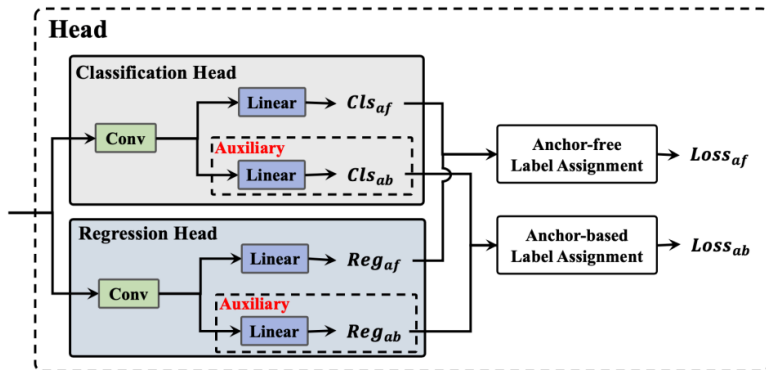


Figure 3: The detection head with anchor-based auxiliary branches during training. The auxiliary branches are removed at inference. ‘af’ and ‘ab’ are short for ‘anchor-free’ and ‘anchor-based’.

Paradigm	AP^{val}	AP^s	AP^m	AP^l
Anchor-free	35.5%	16.0%	39.5%	51.0%
Anchor-based	35.6%	17.2%	39.8%	52.1%

Table 1: Comparisons of the anchor-free and the anchor-based paradigms on YOLOv6-N.

Method	AAT	AP^{val}	AP^s	AP^m	AP^l
YOLOv6-N	✗	36.9%	17.2%	41.1%	52.9%
	✓	36.9%	18.7%	41.2%	53.0%
YOLOv6-S	✗	44.1%	24.7%	48.7%	61.1%
	✓	44.4%	25.4%	49.6%	60.2%
YOLOv6-M	✗	48.6%	29.7%	53.7%	65.5%
	✓	49.1%	31.1%	54.0%	65.4%
YOLOv6-L	✗	51.3%	34.2%	56.5%	67.6%
	✓	51.8%	33.4%	56.8%	68.8%

Table 6: Ablation study about AAT.

YOLO v6 3.0

Self-distillation strategy

Model	Weight Decay	AP^{val}
Baseline	-	51.8%
Double epochs	-	51.7%
Self-distillation	X	51.8%
	✓	52.4%

Table 7: Ablation on the self-distillation on YOLOv6-L

DLD	Double epochs	AP^{val}
X	X	44.4%
X	✓	44.6%
✓	X	45.1%

Table 8: Ablation study of DLD on YOLOv6-S.

YOLO v6 3.0

Experiment and results

Method	Input Size	AP ^{val}	AP ^{val} ₅₀	FPS (bs=1)	FPS (bs=32)	Latency (bs=1)	Params	FLOPs
YOLOv5-N [5]	640	28.0%	45.7%	602	735	1.7 ms	1.9 M	4.5 G
YOLOv5-S [5]	640	37.4%	56.8%	376	444	2.7 ms	7.2 M	16.5 G
YOLOv5-M [5]	640	45.4%	64.1%	182	209	5.5 ms	21.2 M	49.0 G
YOLOv5-L [5]	640	49.0%	67.3%	113	126	8.8 ms	46.5 M	109.1 G
YOLOv5-N6 [5]	1280	36.0%	54.4%	172	175	5.8 ms	3.2 M	18.4 G
YOLOv5-S6 [5]	1280	44.8%	63.7%	103	103	9.7 ms	12.6 M	67.2 G
YOLOv5-M6 [5]	1280	51.3%	69.3%	49	48	20.1 ms	35.7 M	200.0 G
YOLOv5-L6 [5]	1280	53.7%	71.3%	32	30	31.3 ms	76.8 M	445.6 G
YOLOv5-X6 [5]	1280	55.0%	72.7%	17	17	58.6 ms	140.7 M	839.2 G
YOLOX-Tiny [3]	416	32.8%	50.3%*	717	1143	1.4 ms	5.1 M	6.5 G
YOLOX-S [3]	640	40.5%	59.3%*	333	396	3.0 ms	9.0 M	26.8 G
YOLOX-M [3]	640	46.9%	65.6%*	155	179	6.4 ms	25.3 M	73.8 G
YOLOX-L [3]	640	49.7%	68.0%*	94	103	10.6 ms	54.2 M	155.6 G
PPYOLOE-S [17]	640	43.1%	59.6%	327	419	3.1 ms	7.9 M	17.4 G
PPYOLOE-M [17]	640	49.0%	65.9%	152	189	6.6 ms	23.4 M	49.9 G
PPYOLOE-L [17]	640	51.4%	68.6%	101	127	10.1 ms	52.2 M	110.1 G
YOLOv7-Tiny [16]	416	33.3%	49.9%*	787	1196	1.3 ms	6.2 M	5.8 G
YOLOv7-Tiny [16]	640	37.4%*	55.2%*	424	519	2.4 ms	6.2 M	13.7 G*
YOLOv7 [16]	640	51.2%	69.7%*	110	122	9.0 ms	36.9 M	104.7 G
YOLOv7-E6E [16]	1280	56.8%	74.4%*	16	17	59.6 ms	151.7 M	843.2 G
YOLOv8-N [6]	640	37.3%	52.6%*	561	734	1.8 ms	3.2 M	8.7 G
YOLOv8-S [6]	640	44.9%	61.8%*	311	387	3.2 ms	11.2 M	28.6 G
YOLOv8-M [6]	640	50.2%	67.2%*	143	176	7.0 ms	25.9 M	78.9 G
YOLOv8-L [6]	640	52.9%	69.8%*	91	105	11.0 ms	43.7 M	165.2 G

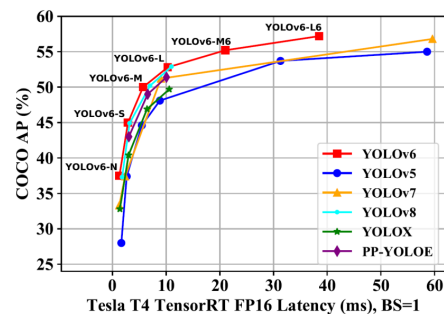
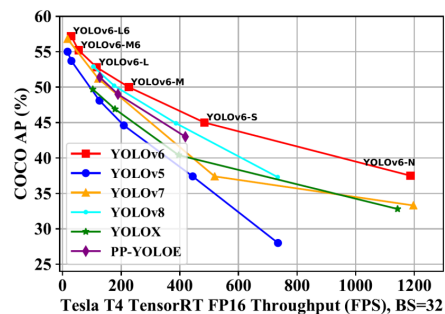


Figure 1: Comparison of state-of-the-art efficient object detectors. Both latency and throughput (at a batch size of 32) are given for a handy reference. All models are test with TensorRT 7.

Method	Input Size	AP ^{val}	AP ^{val} ₅₀	FPS (bs=1)	FPS (bs=32)	Latency (bs=1)	Params	FLOPs
YOLOv6-N	640	37.0% / 37.5% [‡]	52.7% / 53.1% [‡]	779	1187	1.3 ms	4.7 M	11.4 G
YOLOv6-S	640	44.3% / 45.0% [‡]	61.2% / 61.8% [‡]	339	484	2.9 ms	18.5 M	45.3 G
YOLOv6-M	640	49.1% / 50.0% [‡]	66.1% / 66.9% [‡]	175	226	5.7 ms	34.9 M	85.8 G
YOLOv6-L	640	51.8% / 52.8% [‡]	69.2% / 70.3% [‡]	98	116	10.3 ms	59.6 M	150.7 G
YOLOv6-N6	1280	44.9%	61.5%	228	281	4.4 ms	10.4 M	49.8 G
YOLOv6-S6	1280	50.3%	67.7%	98	108	10.2 ms	41.4 M	198.0 G
YOLOv6-M6	1280	55.2% [‡]	72.4% [‡]	47	55	21.0 ms	79.6 M	379.5 G
YOLOv6-L6	1280	57.2% [‡]	74.5% [‡]	26	29	38.5 ms	140.4 M	673.4 G

Conclusion

- YOLO v6 is focused on the industrial application with trading off the performance and speed.
- For the various scenarios, the models have different parameters.
Small model: single-path backbone, Large model: multi-branch block backbone
- Various detection techniques are validated and selectively adopted.
- YOLO v6-v3.0 modifies the network architecture and training scheme.
- For real-time object detection, YOLO v6-v3.0 achieves the SOTA.
- Nowadays, the YOLO's development direction is more commercial than academic.



Question

Thank you